

DISCO: Dependent-type Inspection for String Correctness On-the-fly

Matthew Davis, Denis Koshelev, Pradyumna Shome

1 Abstract

Modern type theory attempts to reject invalid programs based on deviations from sound semantics. Orthogonal to semantics, correct execution of a program also hinges on data flowing through it. Given the weak link in successful software utilization is people [7], it is imperative to ensure inputs and outputs conform to the software specification.

We propose DISCO, a dependently typed string and runtime LLVM compiler pass that detects string formatting issues, such as typos. Our approach can be extended to ensure form fields such as emails, phone numbers, passwords, dates, etc. are valid based on universal formatting conventions. Whenever a string deviates from the expected pattern, the runtime will identify this as a mismatch. This helps reduce usability issues arising from cultural variations in input and output validation.

Building on this, DISCO not only serves as a safeguard against simple errors but also acts as a bridge between user expectations and initial software design. For instance, phone number formats may vary across different cultures and should be accurately validated and formatted. Similarly, a user might encounter different date formats, such as American “MM/DD/YYYY” or the European “DD/MM/YYYY”. Through our validation process, we guarantee that strings with incorrect formatting are identified. This process involves instrumenting wrappers around string I/O functions. As we are focusing on string spelling as a use-case, these wrappers will parse and validate the spelling against a dictionary of terms and then print an error/warning if the string contains misspellings. The developer may use this error report to debug and correct the bug.

We implement a specific form of string validation as a proof of concept in this paper, focused on the spelling of strings. We introduce a grammar and typing rules to allow for this checking, and then implemented these in an LLVM pass which will automatically check the strings of a program prior to output.

2 Introduction

Dependent type detection during runtime can alleviate and detect many issues. Some data requires certain structure or invariants to be held over the course of the program execution. Consider web form validation, where the input of the user needs to match a specific format. If the structure of such data is formalized and a type-checker for it is defined, then there is reduced work for the programmer and a better form for the user that will not miss any edge cases. Another example is cryptographic library validation... and another is in Machine Learning, where some conditions like dynamic checks for acceptable ranges on hyperparameters may help developers debug.

One application that showcases the need for dependent string typing is spellchecking. Many large scale code bases contain spelling errors or grammatical errors in their string literals. It is inconvenient and infeasible for the programmers of the software to manually check the spelling in every string. This manual process is also prone to miss issues or even introduce them. An automated formalism and type-checking for this spellchecking problem can assist developers in correcting their software.

3 Approach

3.1 Spellchecking Formalism

In order to facilitate our specific application of DISCO, we create a formal grammar and typing rules for the words of a string.

String Grammar:

$$\begin{aligned}\langle string \rangle &::= \langle word \rangle \langle string \rangle \mid \langle symbol \rangle \langle string \rangle \mid \epsilon \\ \langle word \rangle &::= \langle alphanum \rangle \langle word \rangle \mid \epsilon \\ \langle alphanum \rangle &::= a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z \mid 0 \mid 1 \mid \dots \mid 9 \\ \langle symbol \rangle &::= , \mid . \mid / \mid : \mid - \mid ' \mid " \mid \backslash \mid [\mid]\end{aligned}$$

This grammar is used to parse out the words of a string, ignoring the symbols.

Typing Rules:

$$\frac{\Gamma \vdash W : \text{WORD} \quad W : \text{WORD}, \Gamma \vdash W \text{ not in } \textit{dictionary}}{\Gamma \vdash W : \text{SPELLCHECKED WORD}} \text{ ty-spellcheck-word}$$
$$\frac{\Gamma \vdash S : \text{STRING} \quad S : \text{STRING}, \Gamma \vdash \forall W : \text{WORD} \in S : \text{SPELLCHECKED WORD}}{\Gamma \vdash S : \text{SPELLCHECKED STRING}} \text{ ty-spellcheck-string}$$

These typing rules are used to typecheck the words of a string to ensure correct spelling. Note that the space on the first grammar construction between word and string is important and intentional.

3.2 Implementation

This section outlines the implementation of DISCO that is segmented into three main components: dictionary generation, spellchecking functionality, and LLVM pass integration.

3.2.1 Dictionary generation

The dictionary generation process is a critical component of the DISCO, which converts a raw text file with replacements for misspelled words into a LLVM-compatible C++ static hash map. The raw dictionary contains pairs of words, the incorrect spelling and its correct form. Our Python code generator parses the dictionary and generates a C++ code that adds all pairs in a hash map that is statically linked in our LLVM pass.

3.2.2 Spellchecking functionality

Our LLVM pass relies on two primary functions: the spellchecking function and the statistics collector. The spellchecking functionality involves parsing C-style input and normalizing it by removing punctuation and converting all characters to lowercase. It then iterates over the tokens, in our case words, performs a lookup in our dictionary, and upon finding a mismatch (if a word is found on the left-hand side), replaces it with the correct version. This function modifies the original string in place. During this process, we also track the number of mismatches that occur in a dedicated global variable, which we print at the end of the LLVM pass.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    puts("I am taking CS 6390: programming slanguages");
    puts("Dr. Sarkar is the profesor of the course");
    puts("We are currently giving a very good presentaion to the class");
    return 0;
}

```

Figure 1: Example program that contains multiple misspellings.

3.2.3 LLVM pass

The final component of DISCO is an actual LLVM pass that invokes the spellchecking functions when necessary. As a first step, we predefine a set of functions whose content we want to spellcheck. In our current implementation, we do this only for the `puts()` function. Before each call to this function, the pass inserts a call to the spellchecking function described above. It checks and corrects the string argument of the output function for spelling errors. Similarly, before a call to any `exit()` function, we insert a function that prints the collected statistics and reports the total number of spelling corrections made during the program’s execution.

4 Results

To test the DISCO we construct a simple C program on Figure 1 that contains several intentional spelling errors within string literals, such as "profesor" instead of "professor", or "programming slanguages" instead of "programming languages". Figure 2 illustrates how we transform our test program to insert calls to our spellchecking and statistics functions in the LLVM IR. First, we generate LLVM IR from the source code. Then, we run our pass to insert calls to our custom functions, and finally, we compile and run the binary. Figure 3 demonstrates the example output of the program described above. The output from running the transformed binary shows that DISCO correctly identified and corrected spelling errors. Each corrected string is preceded by a notification detailing the detected error and suggesting the corrected form. Finally, a summary at the end of the execution reports a total of three spelling errors detected and corrected.

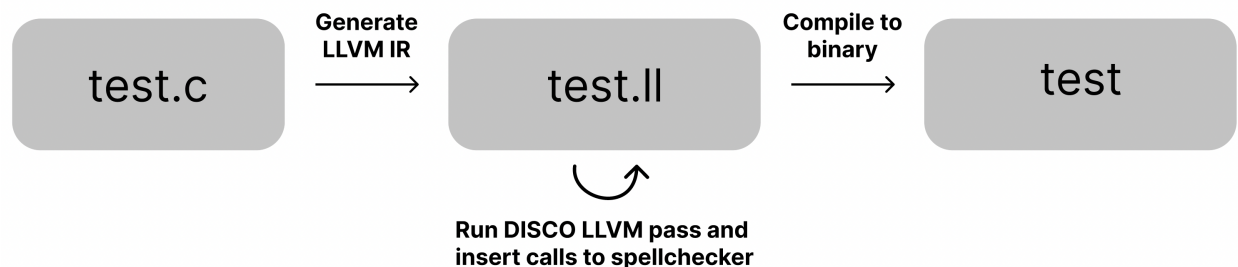


Figure 2: Conversion of the source code to LLVM IR, applying the DISCO pass for spellchecking, and compiling the corrected IR into a new binary.

```

● ~/Desktop/pl/DISCO main ? ..... 8s
  > ./transformed-binary

-----
[DISCO] Detected spelling/grammar mistakes in the upcoming print statement. Probably meant:
I am taking CS 6390: programming languages
-----

I am taking CS 6390: programming slanguages

-----
[DISCO] Detected spelling/grammar mistakes in the upcoming print statement. Probably meant:
Dr. Sarkar is the professor of the course
-----

Dr. Sarkar is the profesor of the course

-----
[DISCO] Detected spelling/grammar mistakes in the upcoming print statement. Probably meant:
We are currently giving a very good presentation to the class
-----

We are currently giving a very good presentaion to the class

[DISCO] 3 spellcheck errors found.

```

Figure 3: The output of the DISCO after termination.

5 Related Work

5.1 Dependent types

Freeman and Pfenning [3] introduced the concept of refinement types which allow developers to add compile-time constraints on user-defined data types in addition to the existing semantics. For example, consider a program trying to extract the last element of a list, which will produce a runtime error when called on an empty list. By specifying this method can only work on non-empty lists, refinement types allow us to get a compile-time error when a particular call can be statically determined to be an empty list.

5.2 Dynamic analysis

There is a rich history of literature proposing code instrumentation for other purposes such as array bounds checking [2], null pointer dereference prevention [1], and sanitizers for undefined behavior [4], uninitialized memory usage [5], and memory leaks [6].

6 Conclusion

In this project, we addressed the critical issue of string formatting errors, which often hinder the correct utilization and adoption of software applications. We proposed a language-based approach to enhance software validation by extending C strings with dependently typed extensions. This approach is particularly beneficial in scenarios requiring precise data formats, such as web forms, public parameter initialization in cryptographic libraries, and hyperparameter clamping in machine learning models.

We implemented our solution through DISCO, an LLVM pass specifically designed to detect and correct spelling errors in input/output operations. This system not only identifies and rectifies errors but also integrates seamlessly into the existing software development workflow. By automating the detection and correction of common input and output errors, DISCO helps bridge the gap between user expectations and software functionality, ensuring that software behaves as intended in a variety of contexts. Our project is an

example of a compiler-level pass that enforces higher-level correctness properties in software systems, paving the way for more robust and error-resistant applications.

7 Future Work

The DISCO effort could be extended in multiple ways.

1. **Support more I/O function call sites:** the implementation of DISCO is currently only supporting the instrumentation of the `puts()` C standard library call. This is a limitation that holds the specific implementation back. If the approach was generalized to handle more or somehow detect all instances where a string is shown to the user and needs to be spellchecked, this could improve the detection.
2. **Cover more string subtypes:** this spellchecking focused DISCO implementation is a proof of concept, but more would need to be done to get this working for other string data like email addresses, phone numbers, etc.
3. **Offer tools to assist developers:** To best use DISCO, we must tightly integrate this with the existing developer tool ecosystem. One could launch a community-driven effort to formalize frequently used string types and implementations to improve adoption. Finally, we should provide means to perform automatically instrument all string output functions, potentially through compiler hints, to extend the value to other languages.

References

- [1] Subarno Banerjee, Lazaro Clapp, and Manu Sridharan. “Nullaway: Practical type-based null safety for java”. In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019, pp. 740–750.
- [2] Rastislav Bodik, Rajiv Gupta, and Vivek Sarkar. “ABCD: eliminating array bounds checks on demand”. In: *Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation*. 2000, pp. 321–333.
- [3] Tim Freeman and Frank Pfenning. “Refinement types for ML”. In: *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*. 1991, pp. 268–277.
- [4] The Chromium Projects. “Undefined Behavior Sanitizer for Chromium.” In: Nov. 2014. URL: <https://www.chromium.org/developers/testing/undefinedbehaviorsanitizer/>.
- [5] Evgeniy Stepanov and Konstantin Serebryany. “MemorySanitizer: fast detector of uninitialized memory use in C++”. In: *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE. 2015, pp. 46–55.
- [6] Yulei Sui, Ding Ye, and Jingling Xue. “Detecting memory leaks statically with full-sparse value-flow analysis”. In: *IEEE Transactions on Software Engineering* 40.2 (2014), pp. 107–122.
- [7] Alma Whitten and J Doug Tygar. “Why Johnny Can’t Encrypt: A Usability Evaluation of PGP 5.0.” In: *USENIX security symposium*. Vol. 348. 1999, pp. 169–184.